МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра квантовых информационных технологий, практической и прикладной информатики

ОТЧЕТ ПО БЛОКУ ПРАКТИЧЕСКИХ РАБОТ №1

По дисциплине

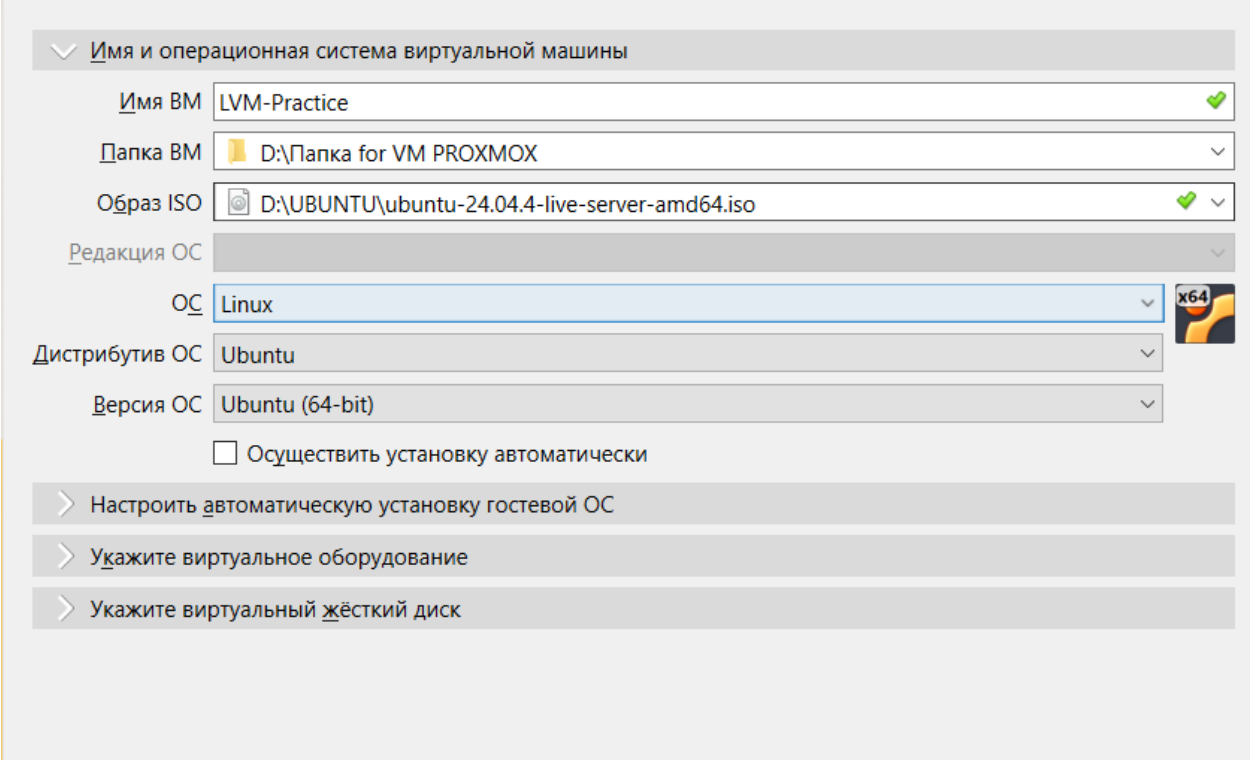
Системы хранения данных

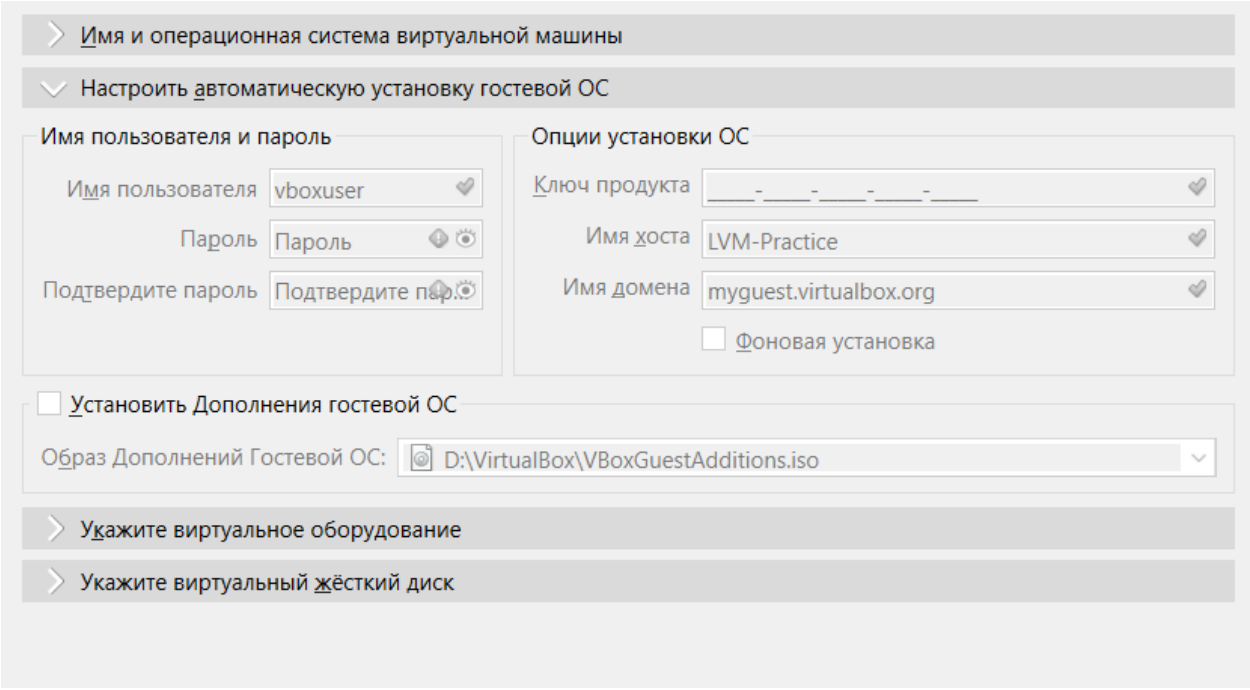
Работа выполнена « »_____2026г. Лияскин.В.А

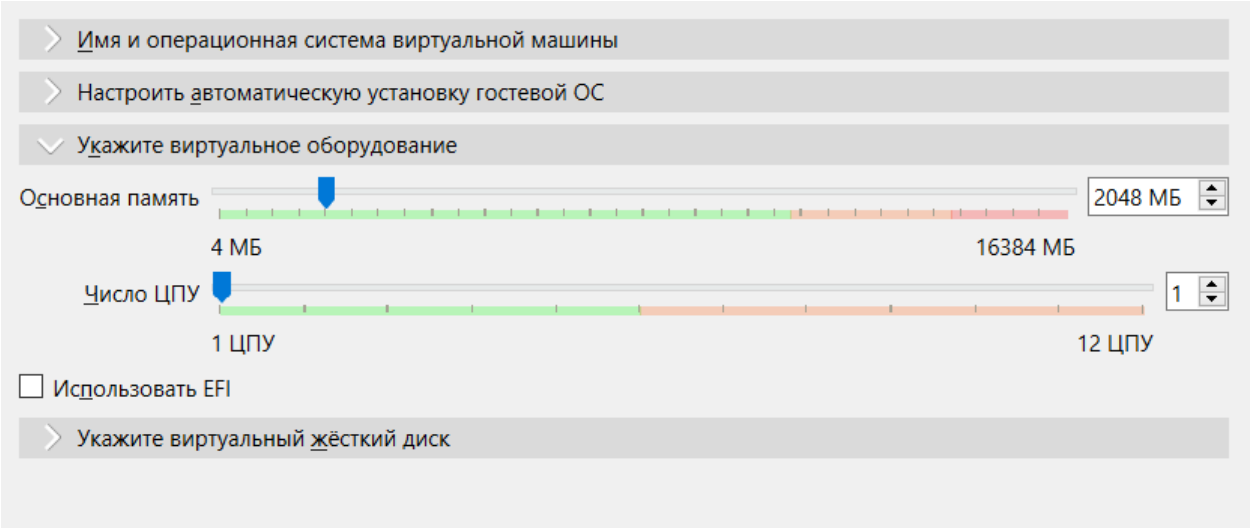
Зачтено « »_____2026г. Павлова Е.С.

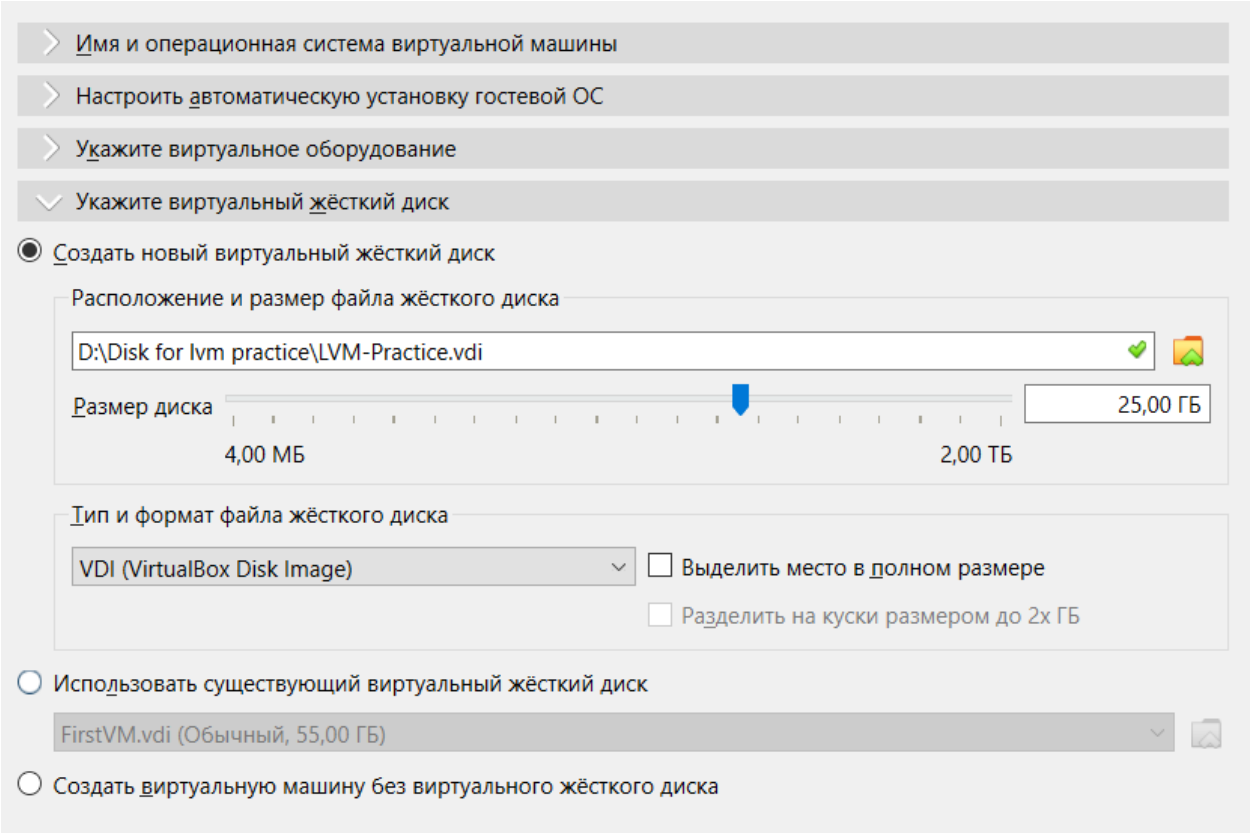
Москва, 2026г.

Создаём ВМ ubuntu server для выполнения практической работы по СХД

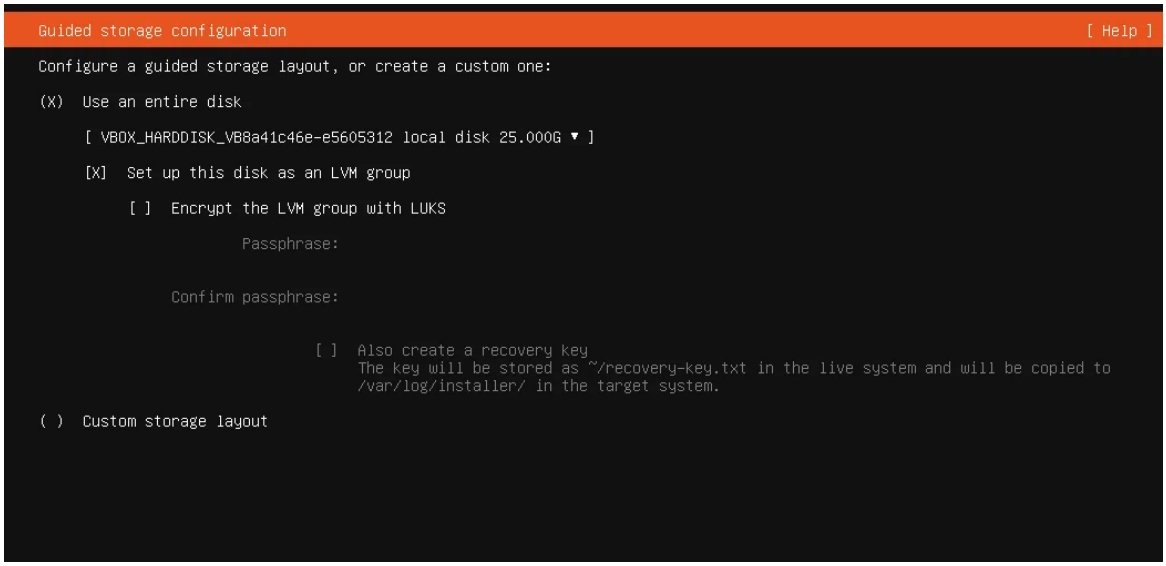




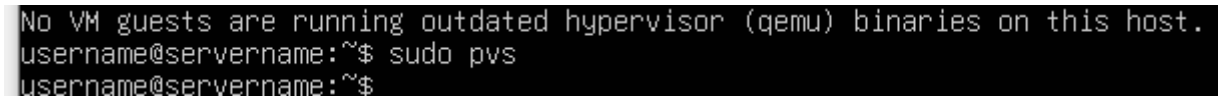

Имя и операционная система виртуальной машины
Настроить автоматическую установку гостевой ОС
Укажите виртуальное оборудование
Основная память
2048 МБ
4 МБ
16384 МБ
Число ЦПУ
1
1 ЦПУ
12 ЦПУ
Использовать EFI
Укажите виртуальный жёсткий диск


Имя и операционная система виртуальной машины
Настроить автоматическую установку гостевой ОС
Укажите виртуальное оборудование
Укажите виртуальный жёсткий диск
Создать новый виртуальный жёсткий диск
Расположение и размер файла жёсткого диска
D:\Disk for lvm practice\LVM-Practice.vdi
Размер диска
25,00 ГБ
4,00 МБ
2,00 ТБ
Тип и формат файла жёсткого диска
VDI (VirtualBox Disk Image)
Выделить место в полном размере
Разделить на куски размером до 2x ГБ
Использовать существующий виртуальный жёсткий диск
FirstVM.vdi (Обычный, 55,00 ГБ)
Создать виртуальную машину без виртуального жёсткого диска

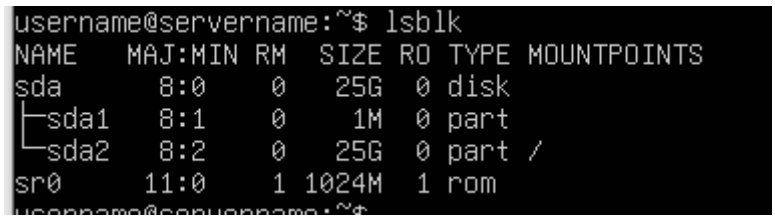
Важный момент



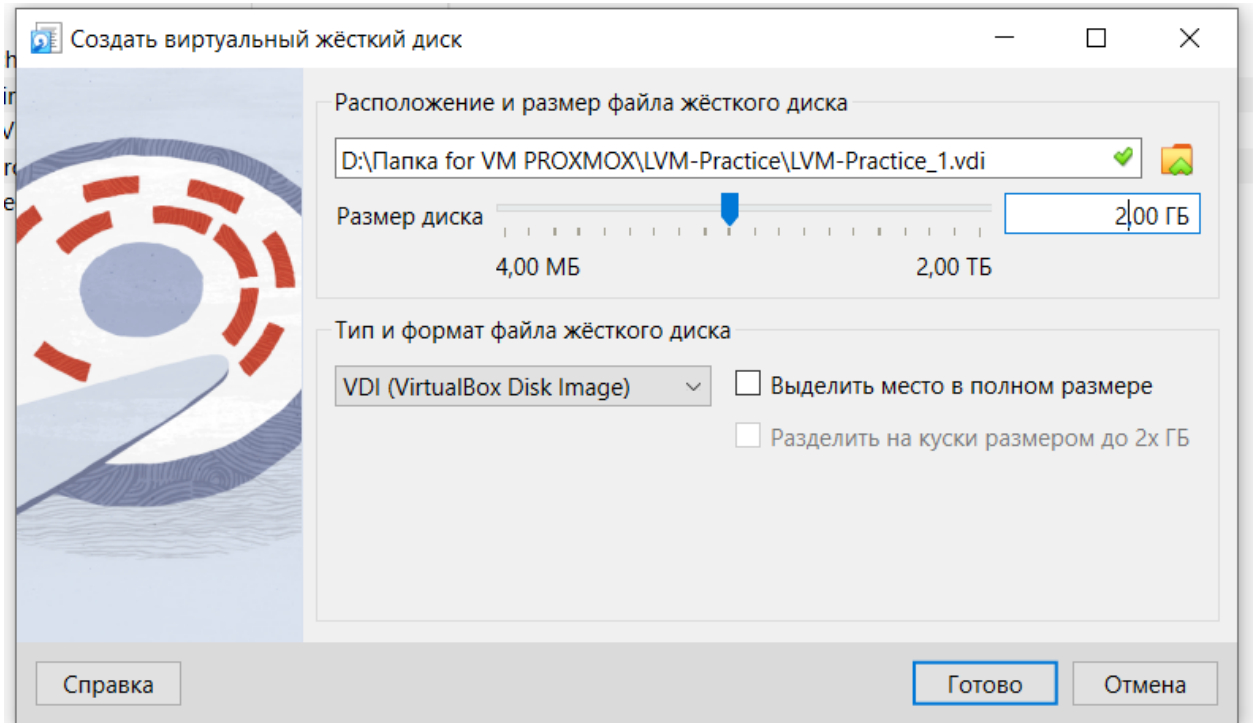
Проверяем, что pvs установлен



Проверяем, что виртуалка видит системный диск

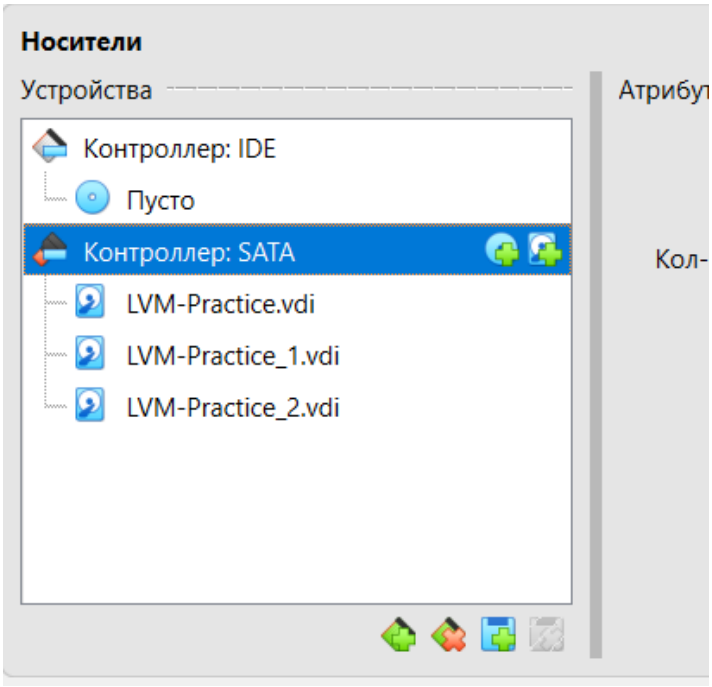


Создаём два диска по 2 гб



Имя	Вирт. размер	Факт. размер
Attached		
FirstVM.vdi	55,00 ГБ	14,76 ГБ
LVM-Practice.vdi	25,00 ГБ	5,26 ГБ
Proxmox (hard drive).vdi	50,00 ГБ	18,92 ГБ
SecondVM.vdi	50,00 ГБ	9,55 ГБ
Not Attached		
LVM-Practice_1.vdi	2,00 ГБ	2,00 МБ
LVM-Practice_2.vdi	2,00 ГБ	2,00 МБ

Подключаем диски к виртуальной машине и проверяем, что они видны в системе




```
username@servername:~$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   25G  0 disk
├─sda1   8:1    0    1M  0 part
└─sda2   8:2    0   25G  0 part /
sdb      8:16   0    2G  0 disk
sdc      8:32   0    2G  0 disk
sr0     11:0    1 1024M  1 rom
username@servername:~$
```

Часть I. Основы работы с физическим томом (PV)

Разметка диска sdb

n (создать новый раздел)

p (primary раздел)

1 (номер раздела)

Enter (начало по умолчанию)

Enter (конец по умолчанию - весь диск)

w (сохранить и выйти)

```
username@servername:~$ sudo fdisk /dev/sdb
[sudo] password for username:

Welcome to fdisk (util-linux 2.39.3).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x4b6cbe0e.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-4194303, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-4194303, default 4194303):

Created a new partition 1 of type 'Linux' and of size 2 GiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

Сообщаем системе об изменениях

```
username@servername:~$ sudo partprobe /dev/sdb
```

Создаем физический том

```
username@servername:~$ sudo pvcreate /dev/sdb1
  Physical volume "/dev/sdb1" successfully created.
```

Проверяем

```
username@servername:~$ sudo pvs
  PV         VG Fmt  Attr PSize  PFree
  /dev/sdb1     lvm2 ---  <2.00g <2.00g
```

Часть II. Группа томов (VG)

Создаем группу томов

```
username@servername:~$ sudo vgcreate vg_liaskin /dev/sdb1
  Volume group "vg_liaskin" successfully created
```

Проверяем

```
username@servername:~$ sudo vgs
  VG         #PV #LV #SN Attr   VSize  VFree
  vg_liaskin   1   0   0 wz--n- <2.00g <2.00g
```

Часть III. Логический том (LV)

Создаем логический том на 1 ГБ

```
username@servername:~$ sudo lvcreate -n lv_liaskin -L 1G vg_liaskin
  Logical volume "lv_liaskin" created.
```

Форматируем в файловую систему ext4

```
username@servername:~$ sudo mkfs.ext4 /dev/vg_liaskin/lv_liaskin
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: 834b699a-913e-41ee-ac83-4a32a7708015
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
```

Смотрим свойства

```
username@servername:~$ sudo lvdisplay /dev/vg_liaskin/lv_liaskin
  --- Logical volume ---
  LV Path                /dev/vg_liaskin/lv_liaskin
  LV Name                lv_liaskin
  VG Name                vg_liaskin
  LV UUID                gw5yID-93gv-kEuL-1h35-2P7L-pFEK-iB9y4a
  LV Write Access        read/write
  LV Creation host, time servername, 2026-09-21 13:24:26 +0000
  LV Status              available
  # open                 0
  LV Size                1.00 GiB
  Current LE             256
  Segments               1
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           252:0
```

Часть IV. Управление размерами логических томов

Расширяем до 1.5 ГБ (добавляем 500 МБ)

```
username@servername:~$ sudo lvextend -L+500M /dev/vg_liaskin/lv_liaskin
  Size of logical volume vg_liaskin/lv_liaskin changed from 1.00 GiB (256 extents) to <1.49 GiB (381 extents).
  Logical volume vg_liaskin/lv_liaskin successfully resized.
username@servername:~$ sudo resize2fs /dev/vg_liaskin/lv_liaskin
resize2fs 1.47.0 (5-Feb-2023)
Resizing the filesystem on /dev/vg_liaskin/lv_liaskin to 390144 (4k) blocks.
The filesystem on /dev/vg_liaskin/lv_liaskin is now 390144 (4k) blocks long.
```

```
username@servername:~$ sudo lvdisplay /dev/vg_liaskin/lv_liaskin
  --- Logical volume ---
  LV Path                /dev/vg_liaskin/lv_liaskin
  LV Name                lv_liaskin
  VG Name                vg_liaskin
  LV UUID                gw5yID-93gv-kEuL-1h35-2P7L-pFEK-iB9y4a
  LV Write Access        read/write
  LV Creation host, time servername, 2026-09-21 13:24:26 +0000
  LV Status              available
  # open                 0
  LV Size                <1.49 GiB
  Current LE             381
  Segments               1
  Allocation             inherit
  Read ahead sectors     auto
  - currently set to     256
  Block device           252:0
```

Уменьшаем обратно до 1 ГБ

Проверяем файловую систему

```
username@servername:~$ sudo e2fsck -f /dev/vg_liaskin/lv_liaskin
e2fsck 1.47.0 (5-Feb-2023)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
/dev/vg_liaskin/lv_liaskin: 11/98304 files (0.0% non-contiguous), 15140/390144 blocks
```

Уменьшаем файловую систему до 1 ГБ

```
username@servername:~$ sudo resize2fs /dev/vg_liaskin/lv_liaskin 1G
resize2fs 1.47.0 (5-Feb-2023)
Resizing the filesystem on /dev/vg_liaskin/lv_liaskin to 262144 (4k) blocks.
The filesystem on /dev/vg_liaskin/lv_liaskin is now 262144 (4k) blocks long.
```

Уменьшаем сам логический том

```
username@servername:~$ sudo lvreduce -L 1G /dev/vg_liaskin/lv_liaskin
  WARNING: Reducing active logical volume to 1.00 GiB.
  THIS MAY DESTROY YOUR DATA (filesystem etc.)
Do you really want to reduce vg_liaskin/lv_liaskin? [y/n]: y
  Size of logical volume vg_liaskin/lv_liaskin changed from <1.49 GiB (381 extents) to 1.00 GiB (256 extents).
  Logical volume vg_liaskin/lv_liaskin successfully resized.
```

Проверяем результат

```
username@servername:~$ sudo lvs
  LV         VG         Attr       LSize Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_liaskin vg_liaskin -wi-a----- 1.00g
```

Часть V. Мониторинг состояния томов

Сканирование томов

```
username@servername:~$ sudo lvscan
  ACTIVE            '/dev/vg_liaskin/lv_liaskin' [1.00 GiB] inherit
```

ACTIVE - текущее состояние тома (активен и готов к использованию)

'/dev/vg_liaskin/lv_liaskin' - полный путь к устройству в системе

[1.00 GiB] - текущий размер логического тома

inherit - политика распределения экстентов (наследует настройки группы томов)

Дополнительное задание:

У меня есть второй свободный диск, поэтому я его размечу на два раздела разного размера

Разметка диска sdc

```
username@servername:~$ sudo fdisk /dev/sdc

Welcome to fdisk (util-linux 2.39.3).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x3d67b64c.

Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): n
Value out of range.
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-4194303, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-4194303, default 4194303): +1.5G

Created a new partition 1 of type 'Linux' and of size 1.5 GiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

n (новый раздел)

p (primary)

1 (номер)

Enter (начало по умолчанию)

+1.5G (размер 1.5 ГБ)

w (сохранить)

Создаем второй раздел на оставшееся место (~500 МБ)

```
username@servername:~$ sudo fdisk /dev/sdc

Welcome to fdisk (util-linux 2.39.3).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help): n
Partition type
   p   primary (1 primary, 0 extended, 3 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (2-4, default 2): 2
First sector (3147776-4194303, default 3147776):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (3147776-4194303, default 4194303):

Created a new partition 2 of type 'Linux' and of size 511 MiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

Сообщаем системе об изменениях

```
username@servername:~$ sudo partprobe /dev/sdc
```

Проверяем разделы

```
username@servername:~$ lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                         8:0    0   25G  0 disk
├─sda1                      8:1    0    1M  0 part
└─sda2                      8:2    0   25G  0 part /
sdb                         8:16   0    2G  0 disk
└─sdb1                      8:17   0    2G  0 part
  └─vg_liaskin-lv_liaskin 252:0    0    1G  0 lvm
sdc                         8:32   0    2G  0 disk
├─sdc1                      8:33   0  1.5G  0 part
└─sdc2                      8:34   0  511M  0 part
sr0                        11:0    1 1024M  1 rom
username@servername:~$
```

Создаем два физических тома из разделов sdc

```
username@servername:~$ sudo pvcreate /dev/sdc1 /dev/sdc2
  Physical volume "/dev/sdc1" successfully created.
  Physical volume "/dev/sdc2" successfully created.
```

Создаем вторую группу томов vg_secondary

```
username@servername:~$ sudo vgcreate vg_secondary /dev/sdc1 /dev/sdc2
  Volume group "vg_secondary" successfully created
```

Создаем два логических тома разного назначения

```
username@servername:~$ sudo lvcreate -n lv_backup -L 1G vg_secondary
  Logical volume "lv_backup" created.
username@servername:~$ sudo lvcreate -n lv_temp -L 500M vg_secondary
  Logical volume "lv_temp" created.
```

Форматируем оба тома в ext4

```
username@servername:~$ sudo mkfs.ext4 /dev/vg_secondary/lv_backup
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: 2e96de1f-4f23-490f-a133-e99c3fd1fc09
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done

username@servername:~$ sudo mkfs.ext4 /dev/vg_secondary/lv_temp
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 128000 4k blocks and 128000 inodes
Filesystem UUID: 90978803-ef57-41dd-a1b8-7f286d4f795c
Superblock backups stored on blocks:
        32768, 98304

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done
```

Проверяем результат

```
username@servername:~$ sudo vgs
  VG           #PV #LV #SN Attr   VSize  VFree
  vg_liaskin     1   1   0 wz--n- <2.00g 1020.00m
  vg_secondary   2   2   0 wz--n-  1.99g  516.00m
username@servername:~$ sudo lvs
  LV         VG           Attr       LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_liaskin vg_liaskin   -wi-a-----   1.00g
  lv_backup  vg_secondary -wi-a-----   1.00g
  lv_temp    vg_secondary -wi-a----- 500.00m
```

Ответы на контрольные вопросы

Блок №1

1. LVM (Logical Volume Manager)- это подсистема управления дисковым пространством в Linux, позволяющая абстрагировать физическое хранилище от файловой системы и гибко управлять томами.
2. Физический том (PV)- это инициализированный для LVM раздел или диск, содержащий метаданные LVM. Физическое устройство хранения-

это "сырое" железо (HDD/SSD). PV- это логическая надстройка над физическим устройством.

3. Группа томов (VG)- это пул хранения, объединяющий один или несколько PV в единое адресное пространство. Используется для гибкого распределения места между логическими томами.
4. Логические тома (LV)- это виртуальные разделы внутри VG, на которых создаются файловые системы. Они заменяют традиционные разделы диска.
5. Да, размер LV можно изменять динамически после создания (расширять и уменьшать), часто без размонтирования ФС.
6. Преимущества LVM: гибкое изменение размеров томов, объединение нескольких дисков в один пул, создание снапшотов, онлайн-миграция данных между дисками, независимость ФС от физической геометрии диска.
7. Физический том создается командой pvcreate.
8. Свойства логического тома отображает команда lvdisplay (краткий вывод- lvs).
9. Да, несколько физических дисков можно объединить в одну группу томов командой vgcreate или vgextend.
10. Жесткого ограничения нет- оно определяется размером метаданных VG и доступным дисковым пространством. Практически можно создать сотни LV.

Блок №2

1. Новый PV добавляется в существующую VG командой: vgextend vg_name /dev/new_pv.
2. Шаги расширения LV:
    - lvextend -L+size /dev/vg/lv- увеличить том
    - resize2fs /dev/vg/lv (для ext4) или xfs_growfs (для xfs)- растянуть ФС

3. Да, разница есть. Расширение можно выполнять "на горячую" (online) без размонтирования. Уменьшение требует: размонтирования ФС → проверки ФС (e2fsck -f) → уменьшения ФС (resize2fs) → уменьшения LV (lvreduce).
4. Удаление PV из VG (vgreduce) возможно только если он не используется LV. Если на PV есть данные- сначала нужно перенести их командой pvmove на другой PV в группе.
5. Команда lvreduce выдаст предупреждение и потребует подтверждения. Если продолжить- данные за пределами нового размера будут безвозвратно потеряны, ФС будет повреждена.
6. Минимальное число PV в группе- один.
7. Очень эффективна: позволяет создавать тома любого размера независимо от физических дисков, легко масштабировать хранилище добавлением новых дисков в VG.
8. Да, все команды LVM (pvcreate, vgcreate, lvcreate и др.) имеют консольный интерфейс и отлично скриптуются на Bash/Python/Ansible.
9. Позволяет переносить данные с деградирующего диска на новый без простоя сервиса (команда pvmove), а также балансировать нагрузку между дисками.
10. Примеры:
    - Закончилось место на разделе- добавили диск в VG и расширили LV без переустановки ОС
    - Нужно сделать бэкап- создали LVM-снапшот за секунды
    - Замена старого HDD на SSD- pvmove переносит данные на лету

Блок №3

1. Концепция гибкости- развязка файловой системы и физического диска. ФС "видит" непрерывное блочное устройство (LV), а LVM может физически хранить его куски на разных дисках, менять их размер и состав без переразметки.

2. Размер PE (Physical Extent): Большой PE (например, 64 МБ) снижает накладные расходы на метаданные и ускоряет работу с большими файлами, но уменьшает точность распределения. Малый PE (4 МБ) даёт гибкость, но увеличивает размер метаданных VG.
3. Проблемы совместимости: Старые ядра Linux могут не поддерживать новые форматы метаданных LVM2 (например, thin provisioning, cache). Обратная совместимость в целом сохраняется, но функции вроде снапшотов новых версий могут быть недоступны на старых ядрах.
4. Алгоритм изменения размера с минимальным воздействием:
    - Проверить свободное место в VG (vgs)
    - Выполнить lvextend -r -L+size /dev/vg/lv (флаг -r автоматически растянет ФС)
    - Мониторить I/O через iostat или iotop в период пиковой нагрузки
    - Выполнять операцию в часы наименьшей нагрузки
5. Меры предосторожности:
    - Делать бэкап метаданных: vgcfgbackup vg_name
    - Проверять ФС перед уменьшением: e2fsck -f
    - Не уменьшать LV ниже фактического размера занятых данных
    - Иметь план отката
6. Разделение PV без потери данных штатными средствами LVM невозможно. Требуется: бэкап данных → уничтожение PV → переразметка диска на два раздела → создание двух новых PV → восстановление данных.
7. LVM избыточен в случаях:
    - Простые домашние ПК с одним диском и фиксированными разделами
    - Специализированные appliance-решения с жёстко заданной конфигурацией
    - Контейнеры и легкие виртуальные машины, где размер диска не меняется

8. Фрагментация в LVM возникает, когда экстенты одного LV разбросаны по разным PV или несмежным областям. Минимизируется:
    - Флагом -C yes (contiguous) при создании LV
    - Дефрагментацией на уровне ФС
    - Правильным планированием размещения PV
9. Snapshot в LVM:
    - *Плюсы:* мгновенный снимок состояния, удобен для бэкапов и тестирования изменений
    - *Минусы:* снижает производительность при активной записи (copy-on-write), не является полноценным бэкапом (зависит от оригинального тома), требует резервирования места
10. Мониторинг и предотвращение деградации:
    - Демон dmeventd для мониторинга событий LVM
    - Интеграция с Prometheus/Grafana (экспортер node_exporter отдаёт метрики node_filesystem_*)
    - Алерты при заполнении VG >80%
    - Регулярный vgdisplay и lvdisplay для контроля состояния
    - SMART-мониторинг физических дисков (smartctl)

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра квантовых информационных технологий, практической и прикладной информатики

ОТЧЕТ ПО БЛОКУ ПРАКТИЧЕСКИХ РАБОТ №2

По дисциплине

Системы хранения данных

Работа выполнена « »_____2026г. Лияскин.В.А

Зачтено « »_____2026г. Павлова Е.С.

Москва, 2026г.

Часть I. Выбор и подготовка файловой системы

Выбрана файловая система Ext4 с размером блока 4096 байт. Ext4 подходит для универсальных задач, поддерживает журналирование (режим ordered), что обеспечивает баланс между производительностью и надежностью. Размер блока 4КБ оптимален для работы с файлами среднего размера и снижает фрагментацию.

Часть II. Форматирование и монтирование

Форматируем логический том в Ext4

```
username@servername:~$ sudo mkfs.ext4 -b 4096 /dev/vg_liaskin/lv_liaskin
[sudo] password for username:
mke2fs 1.47.0 (5-Feb-2023)
/dev/vg_liaskin/lv_liaskin contains a ext4 file system
        created on Mon Sep 21 13:28:07 2026
Proceed anyway? (y,N) y
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: bc6c37a9-d160-41e3-8da3-0f73388a5521
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done
```

Создаем точку монтирования и монтируем

```
username@servername:~$ sudo mkdir -p /mnt/data
username@servername:~$ sudo mount /dev/vg_liaskin/lv_liaskin /mnt/data
```

Проверяем монтирование

```
username@servername:~$ df -hT | grep /mnt/data
/dev/mapper/vg_liaskin-lv_liaskin ext4   974M   24K  907M   1% /mnt/data
```

Создаем тестовый файл

```
username@servername:~$ echo "Tecstoviy file. Name: Liyaskin.V.A, groupe: IVBO-20-24" > test.txt
```

```
username@servername:~$ sudo cp test.txt /mnt/data/
```

Проверяем содержимое

```
username@servername:~$ ls -la /mnt/data/
total 28
drwxr-xr-x 3 root root  4096 Sep 21 17:56 .
drwxr-xr-x 3 root root  4096 Sep 21 17:50 ..
drwx------ 2 root root 16384 Sep 21 17:49 lost+found
-rw-r--r-- 1 root root    55 Sep 21 17:56 test.txt
```

Часть III. Анализ структуры файловой системы

Изучаю inodes

```
username@servername:~$ sudo dumpe2fs /dev/vg_liaskin/lv_liaskin | grep -i inode
```

```
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype needs_recovery extent 64bit flex_bg sparse_super large_file huge_file dir_nlink e
xtra_isize metadata_csum
Inode count:              65536
Free inodes:              65525
Inodes per group:         8192
Inode blocks per group:   512
First inode:              11
Inode size:               256
Journal inode:            8
Journal backup:           inode blocks
  Inode bitmap at 137 (+137), csum 0x887279a1
  Inode table at 145-656 (+145)
  28521 free blocks, 8180 free inodes, 2 directories, 8180 unused inodes
  Free inodes: 13-8192
Group 1: (Blocks 32768-65535) csum 0x930f [INODE_UNINIT, ITABLE_ZEROED]
  Inode bitmap at 138 (bg #0 + 138), csum 0x00000000
  Inode table at 657-1168 (bg #0 + 657)
  32638 free blocks, 8192 free inodes, 0 directories, 8192 unused inodes
  Free inodes: 8193-16384
Group 2: (Blocks 65536-98303) csum 0xc692 [INODE_UNINIT, BLOCK_UNINIT, ITABLE_ZEROED]
  Inode bitmap at 139 (bg #0 + 139), csum 0x00000000
  Inode table at 1169-1680 (bg #0 + 1169)
  32768 free blocks, 8192 free inodes, 0 directories, 8192 unused inodes
  Free inodes: 16385-24576
Group 3: (Blocks 98304-131071) csum 0x7f8d [INODE_UNINIT, BLOCK_UNINIT, ITABLE_ZEROED]
  Inode bitmap at 140 (bg #0 + 140), csum 0x00000000
  Inode table at 1681-2192 (bg #0 + 1681)
  32639 free blocks, 8192 free inodes, 0 directories, 8192 unused inodes
  Free inodes: 24577-32768
Group 4: (Blocks 131072-163839) csum 0xa0cc [INODE_UNINIT, ITABLE_ZEROED]
  Inode bitmap at 141 (bg #0 + 141), csum 0x00000000
  Inode table at 2193-2704 (bg #0 + 2193)
  24576 free blocks, 8192 free inodes, 0 directories, 8192 unused inodes
  Free inodes: 32769-40960
Group 5: (Blocks 163840-196607) csum 0x8d00 [INODE_UNINIT, BLOCK_UNINIT, ITABLE_ZEROED]
  Inode bitmap at 142 (bg #0 + 142), csum 0x00000000
  Inode table at 2705-3216 (bg #0 + 2705)
  32639 free blocks, 8192 free inodes, 0 directories, 8192 unused inodes
  Free inodes: 40961-49152
Group 6: (Blocks 196608-229375) csum 0x53dc [INODE_UNINIT, BLOCK_UNINIT, ITABLE_ZEROED]
  Inode bitmap at 143 (bg #0 + 143), csum 0x00000000
  Inode table at 3217-3728 (bg #0 + 3217)
  32768 free blocks, 8192 free inodes, 0 directories, 8192 unused inodes
  Free inodes: 49153-57344
Group 7: (Blocks 229376-262143) csum 0xc79c [INODE_UNINIT, ITABLE_ZEROED]
  Inode bitmap at 144 (bg #0 + 144), csum 0x00000000
  Inode table at 3729-4240 (bg #0 + 3729)
  32639 free blocks, 8192 free inodes, 0 directories, 8192 unused inodes
  Free inodes: 57345-65536
```

Inode count: 65536 - всего создано inode на разделе

Free inodes: 65525 - свободно (используется только 11)

Inodes per group: 8192 - inode распределены по группам (8 групп по 8192)

Inode size: 256 - размер одного inode в байтах

Journal inode: 8 - inode журнала (зарезервирован)

Journal backup: inode blocks — резервная копия журнала хранится в блоках inode

Проверяем использование inodes

```
username@servername:~$ df -i /mnt/data
Filesystem                         Inodes IUsed IFree IUse% Mounted on
/dev/mapper/vg_liaskin-lv_liaskin  65536    12 65524    1% /mnt/data
```

Изучаем журналирование

```
username@servername:~$ sudo tune2fs -l /dev/vg_liaskin/lv_liaskin | grep -i journal
Filesystem features:      has_journal ext_attr resize_inode dir_index filetype needs_recovery extent 64bit flex_bg sparse_super large_file huge_file dir_nlink e
xtra_isize metadata_csum
Journal inode:            8
Journal backup:           inode blocks
```

Filesystem features: has_journal - журналирование включено

Journal inode: 8 - журнал хранится в inode №8

Journal backup: inode blocks - резервная копия в блоках

Часть IV. Модификация файловой системы

Расширяем LV на 50% (было 1ГБ - станет 1.5ГБ)

```
username@servername:~$ sudo lvextend -L +500M /dev/vg_liaskin/lv_liaskin
  Size of logical volume vg_liaskin/lv_liaskin changed from 1.00 GiB (256 extents) to <1.49 GiB (381 extents).
  Logical volume vg_liaskin/lv_liaskin successfully resized.
```

Растягиваем файловую систему онлайн

```
username@servername:~$ sudo resize2fs /dev/vg_liaskin/lv_liaskin
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/vg_liaskin/lv_liaskin is mounted on /mnt/data; on-line resizing required
old_desc_blocks = 1, new_desc_blocks = 1
The filesystem on /dev/vg_liaskin/lv_liaskin is now 390144 (4k) blocks long.
```

Проверяем новое пространство

```
username@servername:~$ df -hT /mnt/data
Filesystem                         Type  Size  Used Avail Use% Mounted on
/dev/mapper/vg_liaskin-lv_liaskin ext4  1.5G   28K  1.4G   1% /mnt/data
```

Уменьшаем обратно до 1ГБ

Размонтируем файловую систему

```
username@servername:~$ sudo umount /mnt/data
```

Проверяем файловую систему

```
username@servername:~$ sudo e2fsck -f /dev/vg_liaskin/lv_liaskin
e2fsck 1.47.0 (5-Feb-2023)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Checking reference counts
Pass 5: Checking group summary information
/dev/vg_liaskin/lv_liaskin: 12/98304 files (0.0% non-contiguous), 15141/390144 blocks
```

Уменьшаем файловую систему до 1 ГБ

```
username@servername:~$ sudo resize2fs /dev/vg_liaskin/lv_liaskin 1G
resize2fs 1.47.0 (5-Feb-2023)
Resizing the filesystem on /dev/vg_liaskin/lv_liaskin to 262144 (4k) blocks.
The filesystem on /dev/vg_liaskin/lv_liaskin is now 262144 (4k) blocks long.
```

Уменьшаем логический том

```
username@servername:~$ sudo lvreduce -L 1G /dev/vg_liaskin/lv_liaskin
  WARNING: Reducing active logical volume to 1.00 GiB.
  THIS MAY DESTROY YOUR DATA (filesystem etc.)
Do you really want to reduce vg_liaskin/lv_liaskin? [y/n]: y
  Size of logical volume vg_liaskin/lv_liaskin changed from <1.49 GiB (381 extents) to 1.00 GiB (256 extents).
  Logical volume vg_liaskin/lv_liaskin successfully resized.
```

Монтируем обратно

```
username@servername:~$ sudo mount /dev/vg_liaskin/lv_liaskin /mnt/data
```

Проверяем результат

```
username@servername:~$ df -hT /mnt/data
Filesystem                      Type  Size  Used Avail Use% Mounted on
/dev/mapper/vg_liaskin-lv_liaskin ext4  974M   28K  910M   1% /mnt/data
username@servername:~$
```

Часть V. Производительность и совместимость

Тест производительности записи (dd)

```
username@servername:~$ sudo dd if=/dev/zero of=/mnt/data/test_write.img bs=1M count=100 conv=fdatasync
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.206807 s, 507 MB/s
```

Сбрасываем кеш

```
username@servername:~$ echo 3 | sudo tee /proc/sys/vm/drop_caches
3
```

Тест производительности чтения

```
username@servername:~$ sudo dd if=/mnt/data/test_write.img of=/dev/null bs=1M
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0878825 s, 1.2 GB/s
```

Блок №1

1. Файловая система - это способ организации, хранения и именования данных на носителях информации. Определяет структуру файлов, каталогов и метаданных.
2. Популярные ФС в Linux: Ext4, XFS, Btrfs, Ext3, ReiserFS, ZFS, F2FS.
3. Команда для проверки смонтированных ФС: mount, df -hT, cat /proc/mounts.
4. Журналирование - механизм записи изменений в специальный журнал перед фактическим применением, что обеспечивает восстановление после сбоев питания.

5. Точка монтирования - это каталог в дереве каталогов Linux, к которому подключается файловая система устройства.
6. Команда форматирования в ext4: mkfs.ext4 или mke2fs -t ext4.
7. Ext3 vs Ext4: Ext4 поддерживает большие объемы (до 1EB), улучшает производительность (extents вместо block mapping), добавляет journal checksumming, быстрое выделение inode.
8. Inode - структура данных, хранящая метаданные файла (размер, владелец, права, время создания/изменения, указатели на блоки данных). Не хранит имя файла.
9. Проверка свободного места: df -h /mnt/data или df -hT.
10. Определение типа ФС: df -T, mount | grep device, blkid /dev/device.

Блок №2

1. Выбор ФС для СУБД: XFS или Ext4 с размером блока 4-16КБ, режим журналирования ordered или writeback, отключение atime (noatime).
2. Факторы выбора размера блока: размер файлов (большие файлы → большой блок), производительность (больший блок → меньше фрагментация, но больше потерь на мелких файлах).
3. Особенности XFS: высокая производительность на больших файлах, параллельное выделение inode, поддержка файлов до 8EB, online дефрагментация, отличная масштабируемость.
4. Btrfs отличия: copy-on-write, встроенные снапшоты, проверка контрольных сумм, встроенный RAID, онлайн-дефрагментация и изменение размера.
5. Восстановление данных в Ext4: extundelete, testdisk, photorec, восстановление из резервных суперблоков (dumpe2fs, e2fsck -b).
6. Инструменты анализа: dumpe2fs, tune2fs, fsck, e2fsprogs, iostat, iotop, bonnie++, fio.
7. Параметры для high-load: размер блока, режим журналирования, noatime, nodiratime, barrier=0 (если есть батарейка на контроллере), allocator.
8. Online изменение размера: resize2fs (Ext4), xfs_growfs (XFS), lvextend + resize2fs.

9. Offline vs Online: Offline требует размонтирования (безопаснее), Online работает на смонтированной ФС (требует поддержки ФС).
10. Безопасность при изменении: бэкап данных, проверка ФС (fsck), мониторинг свободного места, тестирование в staging.

Блок №3

1. Стратегия выбора ФС для корпоративного хранилища:
    - Анализ нагрузки (чтение/запись, размер файлов)
    - Требования к надежности (journaling, checksums)
    - Масштабируемость (XFS для больших объемов)
    - Btrfs/ZFS для снапшотов и дедупликации
    - Тестирование в production-like среде
2. Этапы миграции ФС:
    - Аудит текущей системы
    - Выбор целевой ФС
    - Создание резервных копий
    - Тестирование миграции
    - План отката
    - Поэтапная миграция
    - Валидация данных
    - Обновление документации
3. Контроль целостности (checksumming):
    - Btrfs/ZFS вычисляют контрольные суммы для данных и метаданных
    - При чтении проверяется сумма → обнаружение silent corruption
    - При наличии RAID - автоматическое восстановление
    - Copy-on-write предотвращает повреждение при записи
4. Сравнение производительности:
    - Ext4: стабильная, хороша для общих задач, средняя производительность

- XFS: лучшая для больших файлов и параллельных операций
- Btrfs: медленнее из-за COW и checksums, но дает дополнительные функции

5. Настройки журналирования для устойчивости:
   - data=ordered (по умолчанию) - баланс надежности и скорости
   - data=journal - максимальная надежность (медленнее)
   - barrier=1 - включение барьеров записи
   - Регулярный fsck при загрузке
6. Аварийное восстановление:
   - Загрузка с Live CD/USB
   - Поиск резервных суперблоков: dumpe2fs /dev/sdX | grep -i superblock
   - Восстановление: e2fsck -b <backup_superblock> /dev/sdX
   - Использование debugfs для ручного восстановления inode
   - Инструменты: testdisk, photorec
7. ZFS достоинства/недостатки:
   - Плюсы: COW, снапшоты, дедупликация, встроенный RAID-Z, checksums, compression, отличная целостность
   - Минусы: требует много RAM, сложность, лицензионные ограничения, не входит в mainline kernel
8. Кластерные ФС для ВМ:
   - OCFS2, GFS2, CephFS
   - Позволяют нескольким хостам одновременно писать
   - Необходимы для live migration, HA кластеров
   - Требуют distributed locking (DLM)
9. Автоматическое отслеживание ошибок:
   - Настройка smartd для мониторинга SMART
   - Скрипты на cron с fsck -n (проверка без исправления)
   - Логирование через dmesg, /var/log/syslog
   - Интеграция с мониторингом (Prometheus + node_exporter)

- Alerting через email/Telegram/Slack

10. Рекомендации для стриминга медиа:
    - XFS - оптимален для больших файлов и последовательного чтения
    - Размер блока 64КБ или больше
    - Отключение atime: noatime,nodiratime
    - Выделение отдельного устройства под журнал
    - Использование RAID 10 или RAID 5 для производительности